Informe de Auditoría

Backend IA · Kohem Chemicals

Laboratorio 4

Campo	Valor
Estudiante	Nombre del Estudiante
Materia	Calidad de Software y Auditoría de Sistemas
Programa	Tecnología de Desarrollo de Software
Institución	Institución Universitaria Pascual Bravo
Versión auditada	Kohem Chemicals MVP · rama lab4-conversacional
Fecha	10/11/2026

Informe de los casos P01–P12 ejecutados sobre el backend del MVP, con resultados, hallazgos y evidencias verificables.

1. Resumen ejecutivo

Se auditó el backend del MVP de Kohem Chemicals, una distribuidora de materias primas, con énfasis en el módulo de inteligencia artificial. El sistema se reconstruyó con un adaptador RAG de políticas comerciales, un servidor MCP real y un motor de lenguaje opcional (DeepSeek), manteniendo la arquitectura Onion del proyecto.

Se ejecutaron los doce casos definidos (P01–P12), que cubren cálculo de pedidos, validación, contrato HTTP, interpretación del lenguaje, reglas RAG, MCP, integración extremo a extremo, errores, arquitectura, motor de lenguaje y modo conversacional. La suite automatizada quedó en 75 pruebas aprobadas y 0 fallos.

Se identificaron cuatro hallazgos abiertos: la ruta de la base de datos depende del directorio de trabajo, la clave JWT conserva un valor por defecto, el servidor MCP no aplica autenticación y el asistente no registra pedidos. El resto de los casos cumple.

El sistema quedó operativo en dos modos: «asistente», con respuestas deterministas y auditables, y «conversacional», donde el modelo actúa como asesor comercial usando el catálogo y las cifras calculadas por el dominio.

Resultados clave

- 12 casos ejecutados, todos en verde.
- 75 pruebas automatizadas aprobadas; 0 fallos.
- 4 hallazgos abiertos documentados con severidad y evidencia.
- Contrato HTTP correcto: los errores de validación devuelven 422/400/415.
- MCP real con tres herramientas y transporte stdio verificado.
- Modo conversacional con historial, catálogo real y cifras ancladas al dominio.

2. Introducción

2.1 Contexto del proyecto

Kohem Chemicals es un Sistema de Información Transaccional para una distribuidora de materias primas: catálogo e inventario, registro de pedidos, asesoría con agente conversacional y auditoría de operaciones. El MVP usa FastAPI, SQLAlchemy asíncrono y SQLite en el backend, y React con Vite en el frontend.

El módulo de IA recibe un mensaje en lenguaje natural, recupera políticas comerciales mediante un adaptador RAG, ejecuta herramientas de catálogo, stock y cotización a través de MCP y devuelve una respuesta estructurada. La arquitectura se organiza en dominio, aplicación, infraestructura y presentación.

2.2 Objetivo del laboratorio

Evaluar la calidad y el comportamiento funcional del backend, verificando la corrección de las reglas de negocio, la validación de solicitudes, el contrato HTTP, la integración entre capas y el comportamiento de los adaptadores RAG y MCP, sin pruebas destructivas ni de carga.

3. Objetivos

Objetivo general

Ejecutar pruebas funcionales y de calidad sobre el backend de Kohem Chemicals, verificando la corrección de las reglas de negocio y la integración entre presentación, aplicación, dominio, RAG y MCP.

Objetivos específicos

- Verificar los cálculos de subtotal, IVA, tope comercial y descuento de stock.
- Evaluar las validaciones de entrada y el tratamiento de errores.
- Comprobar el contrato de la API mediante Swagger y solicitudes HTTP.
- Verificar la comunicación entre el caso de uso y el adaptador MCP, incluido stdio.
- Evaluar la recuperación y aplicación de reglas del adaptador RAG.
- Comprobar la separación de responsabilidades entre las capas Onion.
- Ejecutar las pruebas automatizadas y documentar hallazgos y recomendaciones.

4. Alcance y limitaciones

4.1 Alcance

Se auditaron los componentes del backend relacionados con el módulo de IA y las reglas de pedido:

Componente	Capa	Qué se auditó
app/domain/services.py	Dominio	Subtotal, IVA, tope y stock mínimo.
app/domain/entities.py	Dominio	Entidades puras sin frameworks.
app/application/ports.py	Aplicación	Contratos RAG, MCP y LLM.
app/application/ai_use_cases.py	Aplicación	Orquestación y respuesta.
app/infrastructure/mock_vector_db.py	Infraestructura	Recuperación de políticas.
app/infrastructure/mcp_server.py	Infraestructura	Herramientas y transporte MCP.
app/infrastructure/deepseek_llm.py	Infraestructura	Motor de lenguaje opcional.
app/presentation/validacion.py	Presentación	Validación centralizada.
app/presentation/routers/ai_router.py	Presentación	Contrato de /ai/chat.
tests/ (12 archivos)	Pruebas	Suite automatizada.
docs/laboratorio4/	Evidencias	Scripts, JSON y consolas.

Quedan fuera del alcance el frontend, las pruebas de carga, las pruebas destructivas y la evaluación de la calidad literaria del modelo de lenguaje.

4.2 Limitaciones declaradas

- El RAG es un adaptador en memoria que recupera por palabras clave; no usa embeddings.
- El modo «asistente» no emplea un modelo de lenguaje: usa reglas y plantillas.

- En modo «conversacional» la redacción depende de DeepSeek; sin clave o sin red se devuelve la plantilla del modo asistente.
- El asistente cotiza y consulta stock, pero no registra pedidos ni consulta por ID.
- El servidor no almacena conversaciones: el historial lo envía el cliente.

5. Metodología y entorno

5.1 Ciclo PHVA

Fase	Actividad	Evidencia
Planear	Definir los 12 casos, datos y resultados esperados.	Guía del laboratorio
Hacer	Ejecutar pytest, Swagger y los scripts f01–f12.	evidencias/*.json y *.txt
Verificar	Comparar esperado contra real y clasificar hallazgos.	Este informe
Actuar	Aplicar mejoras y volver a ejecutar la suite.	Evidencias

5.2 Entorno de ejecución

Elemento	Versión / valor
Python	3.14.3
FastAPI	0.143.0
SQLAlchemy	2.1.4 (asíncrono)
Pydantic	2.14.0
SDK MCP	2.3.0
Base de datos	SQLite (kohem.db)
API auditada	http://127.0.0.1:8000
Rama	lab4-conversacional
Etiqueta estable	lab4-asistente-estable

5.3 Evidencias

Cada caso genera un JSON con entradas, salidas y criterios, además de la salida de consola y la salida completa de pytest. Las ejecuciones se conservan en la carpeta evidencias.

6. Arquitectura auditada

Capa	Responsabilidad	Depende de
Dominio	Reglas puras de precio, IVA, tope y stock.	Nada externo
Aplicación	Caso de uso que orquesta RAG, LLM y MCP.	Puertos
Infraestructura	RAG, MCP, DeepSeek y repositorios SQLite.	Dominio y SDK
Presentación	Rutas, validación e inyección de dependencias.	Aplicación

Flujo de una solicitud:

```
POST /api/v1/ai/chat
  → AIAssistantUseCase.respond(message, historial)
```

```
→ IRagRepository.buscar_contexto()
→ ILLMProvider.interpretar() / redactar()      (opcional)
→ IMcpToolExecutor.ejecutar_herramienta()
    → MCPServer (listar_materiales / consultar_stock / calcular_pedido)
        → PricingDomainService + SQLiteRawMaterialRepository
→ AIChatResponseDTO {response, transfer_to_employee, reason, motor, datos}
```

Puertos y adaptadores verificados:

Puerto	Adaptador	Función
IRagRepository	MockVectorDbAdapter	Políticas de IVA, tope y stock.
IMcpToolExecutor	KohemMcpAdapter	Herramientas MCP y transporte stdio.
ILLMProvider	DeepSeekLLMAdapter	Interpretación y redacción natural.

7. Ejecución de los casos

7.1 Matriz de resultados

ID	Caso	Resultado	Evidencia
P01	Operaciones y cálculo	Cumple	f01
P02	Stock y atomicidad	Cumple	f02
P03	Límites y precisión	Cumple	f03
P04	Contrato HTTP	Cumple	f04
P05	Interpretación	10/10	f05
P06	Reglas RAG	Cumple	f06
P07	Adaptador MCP	Cumple	f07
P08	Extremo a extremo	Cumple	f08
P09	Errores	Sin 500	f09
P10	Regresión y arquitectura	Cumple	f10
P11	Motor DeepSeek / reglas	Cumple	f11
P12	Modo conversacional	Cumple	f12

7.2 Detalle por caso

P01 — Dominio y cálculo de pedido

Subtotal 18.000 × 2 = 36.000, total de dos ítems, IVA del 19% y redondeo a dos decimales. La cotización HTTP coincidió con los precios del catálogo. Resultado: cumple.

P02 — Stock insuficiente y atomicidad

La sobreventa se rechaza con 422; un pedido con un ítem sin stock no reserva el primero; la cantidad 0 también devuelve 422.

P03 — Valores límite

Precio negativo, stock negativo y cantidad negativa se rechazan con 422. Los decimales conservan la precisión: 1234,567 × 3 = 3703,70.

P04 — Contrato HTTP y Pydantic

Once escenarios de /ai/chat con códigos 200, 401, 400, 415 y 422. El contraste con /materials también devuelve 422.

P05 — Interpretación del mensaje

Diez variantes: mayúsculas, «2kg», «kilos», números en palabras, dos ítems, negaciones y términos técnicos. Resultado: 10/10.

P06 — Reglas RAG

IVA del 19% con «IVA» e «impuesto»; tope de 50.000.000 solo con términos explícitos; «al máximo» ya no activa la regla. Resultado: cumple.

P07 — Adaptador MCP

Tres herramientas con esquema, rechazo de herramienta inexistente y sesión completa por stdio (initialize → list_tools → call_tool). Resultado: cumple.

P08 — Extremo a extremo

Chat, creación de pedido, descuento de stock, registro en audit-logs, cancelación y restauración del stock. Resultado: cumple.

P09 — Errores y robustez

Prompt vacío, JSON mal formado, tipo de contenido inválido, campos extra, pedido y material inválidos. No hay respuestas 500 ni fugas de trazas.

P10 — Regresión y arquitectura

75 pruebas aprobadas y 0 fallos. El análisis de imports confirma dominio puro y aplicación sin FastAPI, SQLAlchemy ni MCP.

P11 — Motor DeepSeek y respaldo

Con clave, el motor es deepseek o deepseek-conversacional según el modo; sin clave o con fallo de red, responde por reglas. La clave no se registra.

P12 — Modo conversacional

El modelo actúa como asesor con catálogo, políticas e historial; devuelve datos estructurados y mantiene el contrato. Validado con y sin clave.

8. Hallazgos

ID	Hallazgo	Severidad	Estado
H-L4-01	La ruta de SQLite depende del directorio de trabajo.	Baja	Abierto
H-L4-02	SECRET_KEY por defecto sin validación al arrancar.	Media	Abierto
H-L4-03	El servidor MCP no aplica autenticación.	Baja	Abierto
H-L4-04	El asistente no registra pedidos ni consulta por ID.	Informativa	Aceptado

H-L4-01 — Ruta de base de datos dependiente del CWD

DATABASE_URL usa «./kohem.db»; si un script se ejecuta desde otra carpeta, SQLite crea una base vacía y el asistente no encuentra el catálogo.

Evidencia: f07_mcp.json. Recomendación: resolver la ruta contra la raíz del proyecto o exigir una variable absoluta. Estado: abierto.

H-L4-02 — Clave JWT por defecto

SECRET_KEY vale «CHANGE_ME_IN_PRODUCTION» si no existe .env; con ese valor los tokens pueden forjarse.

Evidencia: app/infrastructure/config.py. Recomendación: validar al arrancar y detener el servicio si el valor es el de ejemplo. Estado: abierto.

H-L4-03 — MCP sin autenticación

El servidor MCP no exige credenciales ni distingue roles; cualquier proceso con acceso local puede invocar las herramientas.

Evidencia: f07_mcp.json. Recomendación: mantenerlo en local o añadir token/allowlist. Estado: abierto.

H-L4-04 — Alcance del asistente

El asistente cotiza y consulta stock, pero no crea pedidos ni consulta por identificador.

Evidencia: f08_e2e.json. Es una limitación de alcance, no un defecto. Estado: aceptado.

9. Controles y salvaguardas

Control	Implementación	Dónde se verifica
Validación de entrada	422 para Pydantic, 400 para JSON inválido y 415 para tipo no soportado.	P04, P09
Cifras ancladas	Precios, IVA y totales salen del dominio, no del modelo.	P01, P12
RAG acotado	Coincidencia por palabra completa; el tope exige términos explícitos.	P06
MCP con contrato	Tres herramientas con esquema y errores controlados.	P07
Respaldo del modelo	Si DeepSeek falla o no hay clave, responde el motor de reglas.	P11
Historial limitado	Máximo 8 mensajes enviados por el cliente; no se almacenan.	P12
Pruebas automatizadas	75 pruebas sobre dominio, MCP, IA y HTTP.	P10

Estos controles son los que sostienen la calidad del estado actual del backend y se verifican en los casos indicados.

10. Pruebas automatizadas

Archivo	Pruebas	Qué cubre
test_pricing_domain.py	9	Subtotal, IVA, tope, stock y errores.
test_mcp_adapter.py	11	Esquema, herramientas y errores.
test_ai_use_case.py	10	RAG, cotización y transferencia.
test_ai_presentation.py	9	Contrato HTTP del asistente.
test_validacion_http.py	8	422, 400 y 415 en routers.
test_prompt_mejoras.py	7	Lenguaje natural y negaciones.
test_ai_conversacional.py	7	Modo conversacional e historial.
test_ai_llm.py	6	Interpretación y respaldo.
test_deepseek_adapter.py	4	Cliente, JSON y errores de red.
test_auth.py	2	Registro y token.
test_materials_and_orders.py	1	Permisos y pedido.
test_order_permissions.py	1	Visibilidad de pedidos.

Resultado: 75 pruebas aprobadas, 0 fallos. Persisten tres advertencias de DeprecationWarning por datetime.utcnow, sin impacto funcional.

11. Análisis de arquitectura

El script f10 analiza los imports de cada capa. El dominio no importa FastAPI, SQLAlchemy, MCP ni Pydantic; la aplicación no importa frameworks de infraestructura. Los DTO de entrada y salida usan Pydantic como contrato de borde en presentación, lo cual es aceptable y no contamina el dominio.

Los adaptadores implementan puertos explícitos y se inyectan en el caso de uso. El servidor MCP puede ejecutarse en proceso o por stdio, y el motor de lenguaje es intercambiable sin tocar el dominio ni las reglas de precio.

Riesgo arquitectónico identificado: el repositorio SQLite se instancia con la sesión de la petición y la ruta depende del directorio de trabajo (H-L4-01). No afecta la separación de capas, pero sí la reproducibilidad.

12. Conclusiones

El objetivo del laboratorio se cumplió: los doce casos se ejecutaron con evidencia, los hallazgos se documentaron con severidad y el estado actual del sistema quedó verificado con 75 pruebas automatizadas.

El módulo de IA quedó con una separación de capas correcta, un MCP real y un RAG funcional. La incorporación de DeepSeek mejora la interpretación y permite un modo conversacional con tono de asesor, manteniendo las cifras ancladas al dominio.

Las limitaciones restantes son de alcance y de endurecimiento: ruta de la base, clave JWT, autenticación del MCP y ausencia de registro de pedidos desde el chat.

13. Recomendaciones

- Validar SECRET_KEY al arrancar y rechazar el valor de ejemplo (H-L4-02).
- Resolver DATABASE_URL contra la raíz del proyecto o exigir una ruta absoluta (H-L4-01).
- Añadir autenticación o allowlist al servidor MCP si deja de ser local (H-L4-03).
- Incorporar la herramienta registrar_pedido para cerrar la venta desde el chat (H-L4-04).
- Sustituir el RAG por embeddings y una base vectorial si crece el catálogo.
- Añadir guardarraíles que validen toda cifra de la respuesta contra el bloque de datos.
- Persistir el historial si se requiere continuidad entre sesiones.
- Medir coste y latencia por conversación antes de un despliegue real.

14. Referencias

FastAPI. Documentación oficial. https://fastapi.tiangolo.com
SQLAlchemy. Async ORM. https://docs.sqlalchemy.org
Model Context Protocol. Especificación. https://modelcontextprotocol.io
DeepSeek. API compatible con OpenAI. https://api-docs.deepseek.com
Pydantic. Validación de datos. https://docs.pydantic.dev

Anexo A. Índice de evidencias

Archivo	Contenido
f01_operaciones.json	Dominio y cotización.
f02_stock.json	Sobreventa, atomicidad y cantidad cero.
f03_limites.json	Negativos, decimales y redondeo.
f04_contrato.json	Once escenarios del contrato HTTP.
f05_prompt.json	Diez variantes de lenguaje natural.
f06_rag.json	Contexto recuperado y reglas aplicadas.
f07_mcp.json	Esquema de herramientas y stdio.
f08_e2e.json	Chat, pedido, stock y auditoría.
f09_errores.json	Matriz de errores y fugas.
f10_regresion.json	Suite y análisis de capas.
f10_pytest.txt	Salida completa de pytest.
f11_deepseek.json	Motor usado y respaldo.
f12_conversacional.json	Modo conversacional e historial.
evidencias/	JSON y consolas de la ejecución actual.

Anexo B. Correspondencia con la guía original

Guía original (calculadora)	Adaptación en Kohem Chemicals
Cuatro operaciones básicas	PricingDomainService: subtotal, total, IVA y tope
División por cero	Stock insuficiente y consistencia transaccional
calculator_use_case.py	ai_use_cases.AIAssistantUseCase
RAG con impuesto 15% y tope 1000	IVA 19%, tope 50.000.000 y stock mínimo 10 kg
mcp_server con la herramienta calcular	Kohem-Chemicals-MCP con tres herramientas
Componente LLM simulado	Reglas por defecto y DeepSeek opcional
POST /api/v1/calcular	POST /api/v1/ai/chat
Puerto 7000	Puerto 8000
Respuesta CalculoResult	AIChatResponseDTO con motor y datos

Anexo C. Reproducción

Requisitos y comandos:

```
cd "C:\Auditoria\KohemChemicals\1. Diagnostico Auditoria\MVP"
.\venv\Scripts\python.exe -m pip install -r requirements.txt
.\venv\Scripts\python.exe -m uvicorn app.presentation.main:app --reload
.\venv\Scripts\python.exe -m pytest -q
```

Scripts de evidencia (desde docs\laboratorio4):

```
..\..\venv\Scripts\python.exe f01_operaciones.py
...
..\..\venv\Scripts\python.exe f12_conversacional.py
```

Motor de lenguaje:

La clave se configura en el archivo .env del MVP, que no se versiona. Variables: DEEPSEEK_API_KEY, DEEPSEEK_BASE_URL, DEEPSEEK_MODEL, AI_MODE y AI_HISTORY_MAX. Sin clave, el sistema funciona con reglas.

Anexo D. Ejemplo de conversación en modo conversacional

Transcripción resumida de una ejecución real con DeepSeek (motor deepseek-conversacional).

Cliente: Hola

Asistente: ¡Hola! Bienvenido a Kohem Chemicals. Trabajamos materias primas para cosmética y aseo; tenemos disponibles aceite esencial de menta, glicerina vegetal y ácido cítrico. ¿Qué tipo de producto estás desarrollando?

Cliente: ¿Qué venden?

Asistente: Manejamos materias primas para cosmética y aseo. Hoy tenemos: aceite esencial de menta ($85.000/kg, 50 kg), glicerina vegetal ($18.000/kg, 100 kg) y ácido cítrico ($12.000/kg, 75 kg). ¿Qué producto tienes en mente?

Cliente: ¿Y cuánto cuesta la glicerina?

Asistente: La glicerina vegetal está a $18.000 por kilo y tenemos 100 kg disponibles. Si me dices cuántos kilos necesitas, te armo la cotización completa. ¿Para qué aplicación la vas a usar?